# MO89905: Homework 2

Md. Symum Hossain (ID: 20264789)
Cho Chun Shik Graduate School of Mobility,
Korea Advanced Institute of Science and Technology (KAIST)

Fall 2026

## Problem 1 (Exact policy iteration)

### Part-1: Finding the admissible controls

*Need to list $U(e)$ for each state and explain the all successors remain in $\mathcal{X}$ without clipping or interpolation.*

The given control set:

$$U(e) = \{u \in \{-1, 0, 1\} : |e + u| \leq 3\}.$$

As we can see the controller has three possible actions:

$$u \in \{-1, 0, 1\}.$$

Among three different action, the controller cannot always use all three actions at the same time. Also the action must satisfy the constant as per the specified condition:

$$|e + u| \leq 3. \tag{1}$$

The state transition is

$$e_{k+1} = e_k + u_k + w_k. \tag{2}$$

Before the random disturbance $w_k$ is added, the controlled part of the next state is $e_k + u_k$. The constraint requires this intermediate quantity to stay between $-3$ and 3. This is why specified condition [1] is less or equal 3.

Since the disturbance $w_k$ can be $-1$, 0, or 1, this condition ensures that the next state stays within the defined state space $\{-4, \ldots, 4\}$.

We have nine possible states:

$$e \in \{-4, -3, -2, -1, 0, 1, 2, 3, 4\}. \tag{3}$$

We test each possible control $u = -1, 0, 1$ and retain only those satisfying $|e + u| \leq 3$ in [1].

Now lets evaluate the control set for each possibilities:- If we consider $e = -4$.

- If $u = -1$, then $|-4 - 1| = 5$, which violates the constraint.
- If $u = 0$, then $|-4| = 4$, which violates the constraint.
- If $u = 1$, then $|-4 + 1| = 3$, which satisfies the constraint.

Therefore,

$$U(-4) = \{1\}.$$

The controller must choose $u = 1$ at state $-4$.

We must repeat this process for all nine states defined in equation [3]. Now repeating the state and evaluating and oganizing as below using a table.

Table 1: 9-state Admissible-Control Table

| **Current state** $e$ | $u=-1$**:** $\lvert e+u\rvert$ | $u=0$**:** $\lvert e+u\rvert$ | $u=1$**:** $\lvert e+u\rvert$ | **Admissible set** $U(e)$ |
|---|---|---|---|---|
| $-4$ | 5 — No | 4 — No | 3 — Yes | $\{1\}$ |
| $-3$ | 4 — No | 3 — Yes | 2 — Yes | $\{0,1\}$ |
| $-2$ | 3 — Yes | 2 — Yes | 1 — Yes | $\{-1,0,1\}$ |
| $-1$ | 2 — Yes | 1 — Yes | 0 — Yes | $\{-1,0,1\}$ |
| 0 | 1 — Yes | 0 — Yes | 1 — Yes | $\{-1,0,1\}$ |
| 1 | 0 — Yes | 1 — Yes | 2 — Yes | $\{-1,0,1\}$ |
| 2 | 1 — Yes | 2 — Yes | 3 — Yes | $\{-1,0,1\}$ |
| 3 | 2 — Yes | 3 — Yes | 4 — No | $\{-1,0\}$ |
| 4 | 3 — Yes | 4 — No | 5 — No | $\{-1\}$ |

The table shows, the farthest column is explaining the list of $U(e)$.

As you can see, at the extreme state $-4$, only the positive control is allowed.
At the other extreme, 4, only the negative control is allowed.
and at Near the center, all three controls are admissible.

**The answer of why do all successors remain in $\mathcal{X}$ without clipping or interpolating is because of the design of the problem.**

The state transition in equation in equation [2] where

$$w_k \in \{-1,0,1\}.$$

The admissibility constraint guarantees that

$$|e_k + u_k| \leq 3.$$

or,

$$-3 \leq e_k + u_k \leq 3.$$

Now considering the disturbance where the smallest possible value is $-1$, and its largest possible value is 1.

Therefore, the smallest possible successor is

$$e_{k+1}^{\min} = -3 + (-1) = -4,$$

and the largest possible successor is

$$e_{k+1}^{\max} = 3 + 1 = 4.$$

And therefore,

$$-4 \leq e_{k+1} \leq 4.$$

Because the state and control are integers and the disturbance is an integer, every successor is also an integer. Hence,

$$e_{k+1} \in \{-4,-3,\ldots,3,4\} = \mathcal{X}.$$

This is why no need clipping or interpolation. (Proved)

## Part-2: Exact policy evaluation and greedy policy improvement

Implementing the exact policy evaluation and greedy policy improvement. Evaluating policy by solving $(I-\gamma P_\mu)J_\mu = g_\mu$ and Repeating until the policy does not change.

Lets define the states, controls, disturbance, transition, cost, and discount factor before writing the algorithm. State space $X$

$$X = \{-4,-3,-2,-1,0,1,2,3,4\}.$$

There are nine possible states. The state $e$ represents the lateral position error in the lane-keeping model defined as $e = \{-4,-3\cdots+3,+4\}$. The admissible action set is

$$U(e) = \{u \in \{-1,0,1\} : |e+u| \leq 3\}.$$

The transition is now

$$e_{k+1} = e_k + u_k + w_k$$

Disturbance $w$ and its probabilities The disturbance is the random change affecting the next state:

$$w \in \{-1, 0, 1\}.$$

Its probabilities are

$$P(w = -1) = 0.10, \qquad P(w = 0) = 0.60, \qquad P(w = 1) = 0.30.$$

Stage cost is $g(e, u)$
The immediate cost is

$$g(e, u) = e^2 + 0.20u^2.$$

There are two components: $e^2$ = penalizes lateral error, $0.20u^2$ = penalizes steering effort. The squared error means large deviations are penalized more heavily than small ones.

Discount factor $\gamma$ The discount factor is

$$\gamma = 0.95.$$

It determines how much future costs contribute to the present value.

- initial: The current cost has weight 1.
- The next-step cost has discount weight 0.95.
- A cost two steps into the future has weight $0.95^2$.

Because $0 < \gamma < 1$, distant future costs receive progressively smaller weights. This also helps ensure that the discounted infinite-horizon value remains finite for this bounded-cost model.
*Initial policy:* Lets start with the feasible policy

$$\mu_0 = [1, 0, 0, 0, 0, 0, 0, 0, -1].$$

Here except the extreme point where the control action must satisfy the admissible actions and for the other middle options the zero or center control selected for the simplicity of the problem flow.

**System of Linear (Matrix) Formulation:-** The value function $J_\mu(e)$ The value function Mathematically,

$$J_\mu(e) = \mathbb{E}^\mu \left[ \sum_{k=0}^{\infty} \gamma^k g(e_k, \mu(e_k)) \,\middle|\, e_0 = e \right].$$

Where,

- $J_\mu(e)$: the value of starting in state $e$.
- $\mathbb{E}^\mu$: the expectation when following policy $\mu$ and accounting for random disturbances.
- $g(e_k, \mu(e_k))$: the cost incurred at time $k$.
- $\gamma^k$: the discount applied to the cost at time $k$.
- $e_0 = e$: the initial state is $e$.

Separate the current cost from all future costs:

$$J_\mu(e) = \mathbb{E}^\mu \left[ g(e_0, \mu(e_0)) + \sum_{k=1}^{\infty} \gamma^k g(e_k, \mu(e_k)) \,\middle|\, e_0 = e \right].$$

Since $e_0 = e$, the first cost is $g(e, \mu(e))$. Also, factor $\gamma$ out of the future-cost sum:

$$J_\mu(e) = g(e, \mu(e)) + \gamma \mathbb{E}^\mu \left[ \sum_{k=1}^{\infty} \gamma^{k-1} g(e_k, \mu(e_k)) \,\middle|\, e_0 = e \right].$$

The expression inside the expectation is the future cost starting from the next state $e_1$. Thus,

$$J_\mu(e) = g(e, \mu(e)) + \gamma \mathbb{E}^\mu [J_\mu(e_1) \mid e_0 = e].$$

The next state is determined by the current state, chosen action, and disturbance:

$$e_1 = e + \mu(e) + w.$$

evaluating a policy by solving

$$(I - \gamma P_\mu) J_\mu = g_\mu.$$

Each symbol has a specific meaning. Symbol Meaning $I$ $9 \times 9$ identity matrix, $\gamma$ Discount factor 0.95, $P_\mu$ Transition probability matrix under policy $\mu$, $J_\mu$ Nine-element vector of state values, $g_\mu$ Nine-element vector of immediate costs under policy $\mu$,

The policy value vector is (initial case)

$$J_\mu = \begin{bmatrix} J_\mu(-4) \\ J_\mu(-3) \\ J_\mu(-2) \\ J_\mu(-1) \\ J_\mu(0) \\ J_\mu(1) \\ J_\mu(2) \\ J_\mu(3) \\ J_\mu(4) \end{bmatrix}.$$

The immediate cost vector is (initial case)

$$g_\mu = \begin{bmatrix} g(-4, \mu(-4)) \\ g(-3, \mu(-3)) \\ \vdots \\ g(4, \mu(4)) \end{bmatrix}.$$

The nine Bellman equations can be combined into

$$J_\mu = g_\mu + \gamma P_\mu J_\mu.$$

Move the future-value term to the left:

$$J_\mu - \gamma P_\mu J_\mu = g_\mu.$$

Factor out $J_\mu$:

$$(I - \gamma P_\mu) J_\mu = g_\mu.$$

This is a system of nine linear equations with nine unknown values. Solving it gives the exact value function for the fixed policy.

Need to apply code to evaluate the function.

**Greedy policy improvement**

After evaluating the current policy, we have $J_\mu(e)$ for every state. We now use those values to see whether another action would give a lower cost. 5.1 Calculate the action value $Q_\mu(e, u)$ For every admissible action $u \in U(e)$, calculate

$$Q_\mu(e, u) = g(e, u) + \gamma \sum_{w=-1}^{1} P(w) J_\mu(e + u + w).$$

This is the one-step cost plus discounted expected future cost.

It differs from $J_\mu(e)$ in one important way: - $J_\mu(e)$ evaluates the action already chosen by the current policy and then follows that policy into the future. - $Q_\mu(e,u)$ lets us test a candidate action $u$ now, then uses the current value function $J_\mu$ to estimate the future cost. We calculate $Q_\mu(e,u)$ for every admissible action and choose the action with the lowest result:

$$\mu_{\text{new}}(e) = \arg\min_{u \in U(e)} Q_\mu(e,u).$$

The notation arg min means "the action $u$ that gives the minimum value." The important principle is that we compare expected total cost, not just immediate cost. An action with a small immediate cost can still be worse if it leads to states with high future costs. **The improvement step must consider only $u \in U(e)$.**

**Stopping Criterion** Repeat until the policy stops changing We now have the two necessary functions:

- $policy_evaluation(policy) calculates the current value function.$
- $greedy_improvement(J) calculates the improved policy.$

The outer loop repeats these two steps. Comparing the old and new policies until to consecutive policies remain exactly the same without implying any change.

**Algorithm 1:** Exact Policy Iteration

**Input :** State space $X$, admissible action sets $U(e)$, disturbance probabilities $P(w)$, cost $g(e,u)$, discount factor $\gamma$.

**Output:** Optimal policy $\mu^*$ and value function $J^*$.

`// Step 1: Initialize`

Set $\mu_0 = [1,0,0,0,0,0,0,0,-1]$

// except extreme point rest are chosen based on the simplicity

**repeat**

`// Step 2: Policy Evaluation`

Construct $P_\mu$ and $g_\mu$

Solve the linear system:

$$(I - \gamma P_\mu) J_\mu = g_\mu$$

`// Step 3: Policy Improvement`

**for** *each* $e \in X$ **do**

Update the policy:

$$\mu_{\text{new}}(e) = \arg\min_{u \in U(e)} \left[ g(e,u) + \gamma \sum_w P(w) J_\mu(e+u+w) \right]$$

*Note: Resolve ties using tolerance* $10^{-12}$ *and action order* $-1, 0, 1$.

**end**

`// Step 4: Convergence Check`

**if** $\mu_{\text{new}} == \mu$ **then**

$\mu^* = \mu$

$J^* = J_\mu$

**return** $\mu^*, J^*$

**end**

**else**

$\mu = \mu_{\text{new}}$

**end**

**until** $\mu_{\text{new}} = \mu$

```
PS E:\KAIST\Learning for Control Mobility\hw_02-Learning_for_Control> & C:\Users\symum\AppData\Local\Python\pythoncore-3.14
exe "e:/KAIST/Learning for Control Mobility/hw_02-Learning_for_Control/Problem_One_Part_Two"

Iteration 0
Evaluated policy: [1, 0, 0, 0, 0, 0, 0, 0, -1]
Value function: [118.009897, 110.809897, 96.271283, 94.279689, 106.647348, 129.479963, 156.297845, 178.622726, 185.822726]
Mean state value: 130.693486
Improved policy: [1, 1, 0, -1, -1, -1, -1, -1, -1]
Number of changed actions: 6

Iteration 1
Evaluated policy: [1, 1, 0, -1, -1, -1, -1, -1, -1]
Value function: [91.011713, 79.12, 73.92, 71.12, 66.798322, 64.379641, 66.073057, 74.094697, 90.50478]
Mean state value: 75.22469
Improved policy: [1, 1, 1, 1, 1, 0, -1, -1, -1]
Number of changed actions: 4

Iteration 2
Evaluated policy: [1, 1, 1, 1, 1, 0, -1, -1, -1]
Value function: [58.616212, 45.484917, 38.320244, 35.51337, 35.92, 36.72, 39.92, 49.57035, 67.47902]
Mean state value: 45.282679
Improved policy: [1, 1, 1, 0, 0, -1, -1, -1, -1]
Number of changed actions: 3

Iteration 3
Evaluated policy: [1, 1, 1, 0, 0, -1, -1, -1, -1]
Value function: [38.708215, 24.772774, 16.527907, 13.327907, 11.090698, 12.290698, 17.145894, 28.168917, 47.391808]
Mean state value: 23.269424
Improved policy: [1, 1, 1, 1, 0, -1, -1, -1, -1]
Number of changed actions: 1

Iteration 4
Evaluated policy: [1, 1, 1, 1, 0, -1, -1, -1, -1]
Value function: [36.321852, 22.241443, 14.012818, 10.32, 9.12, 10.32, 15.313007, 26.445894, 45.774678]
Mean state value: 21.096632
Improved policy: [1, 1, 1, 1, 0, -1, -1, -1, -1]
Number of changed actions: 0

========== FINAL RESULTS ==========
States: [-4, -3, -2, -1, 0, 1, 2, 3, 4]
Optimal policy: [1, 1, 1, 1, 0, -1, -1, -1, -1]
Optimal value function: [36.321852, 22.241443, 14.012818, 10.32, 9.12, 10.32, 15.313007, 26.445894, 45.774678]
Mean optimal state value: 21.096632
Policy is admissible at every state: True
PS E:\KAIST\Learning for Control Mobility\hw_02-Learning_for_Control>
```

Figure 1: Results

*Click for Code*

Table 2: Policy Iteration Metrics and Convergence Summary

| Iteration | Vector Type | Values / Actions per State $[-4, \ldots, 4]$ | Mean Value | Changes |
|---|---|---|---|---|
| **Iteration 0** | Evaluated Policy | $[1, 0, 0, 0, 0, 0, 0, 0, -1]$ | 130.693 | 6 |
| | Value Function | $[118.01, 110.81, 96.27, 106.65, 129.50, 156.30, 178.62, 185.82, 185.82]$ | | |
| | Improved Policy | $[1, 1, 0, -1, -1, -1, -1, -1, -1]$ | | |
| **Iteration 1** | Evaluated Policy | $[1, 1, 0, -1, -1, -1, -1, -1, -1]$ | 75.225 | 4 |
| | Value Function | $[91.01, 79.12, 73.92, 71.12, 66.80, 64.38, 66.07, 74.09, 90.50]$ | | |
| | Improved Policy | $[1, 1, 1, 1, 1, 0, -1, -1, -1]$ | | |
| **Iteration 2** | Evaluated Policy | $[1, 1, 1, 1, 1, 0, -1, -1, -1]$ | 45.283 | 3 |
| | Value Function | $[58.62, 45.48, 38.32, 35.51, 35.92, 36.72, 39.92, 49.57, 67.48]$ | | |
| | Improved Policy | $[1, 1, 1, 0, 0, -1, -1, -1, -1]$ | | |
| **Iteration 3** | Evaluated Policy | $[1, 1, 1, 0, 0, -1, -1, -1, -1]$ | 23.269 | 1 |
| | Value Function | $[38.71, 24.77, 16.53, 13.33, 11.09, 12.29, 17.15, 28.17, 47.39]$ | | |
| | Improved Policy | $[1, 1, 1, 1, 0, -1, -1, -1, -1]$ | | |
| **Iteration 4 (Optimal)** | Evaluated Policy | $[1, 1, 1, 1, 0, -1, -1, -1, -1]$ | 21.097 | 0 |
| | Value Function | $[36.32, 22.24, 14.01, 10.32, 9.12, 10.32, 15.31, 26.45, 45.77]$ | | |
| | Improved Policy | $[1, 1, 1, 1, 0, -1, -1, -1, -1]$ | | |

## Part-3: Optimal Policy and Optimal Value Function

As per the evaluation, and table-[8] we can determine that, the system converged to the following policy:

$$\mu^* = [1, 1, 1, 1, 0, -1, -1, -1, -1]$$

and the entries correspond to the states (e) in this order:

$$[-4, -3, -2, -1, 0, 1, 2, 3, 4].$$

and Optimal value function for all nine states is followings, the converged value function is

$$J^* = \begin{bmatrix} 36.321852 \\ 22.241443 \\ 14.012818 \\ 10.320000 \\ 9.120000 \\ 10.320000 \\ 15.313007 \\ 26.445894 \\ 45.774678 \end{bmatrix}$$

and the entries correspond to the states from $-4, -3, \ldots, 2, 3, 4$. The value function measures the expected total discounted cost.

So, **the number of improvement iterations that change the policy (one count per iteration is shown below,**

Table 3: Policy Iteration Evaluation and Actions

| Iteration | Evaluated Policy | Improved Policy | Changed Actions |
|---|---|---|---|
| 0 | $\mu_0 = [1, 0, 0, 0, 0, 0, 0, 0, -1]$ | $[1, 1, 0, -1, -1, -1, -1, -1, -1]$ | 6 |
| 1 | $\mu_1 = [1, 1, 0, -1, -1, -1, -1, -1, -1]$ | $[1, 1, 1, 1, 1, 0, -1, -1, -1]$ | 4 |
| 2 | $\mu_2 = [1, 1, 1, 1, 1, 0, -1, -1, -1]$ | $[1, 1, 1, 0, 0, -1, -1, -1, -1]$ | 3 |
| 3 | $\mu_3 = [1, 1, 1, 0, 0, -1, -1, -1, -1]$ | $[1, 1, 1, 1, 0, -1, -1, -1, -1]$ | 1 |
| 4 | $\mu_4^* = [1, 1, 1, 1, 0, -1, -1, -1, -1]$ | $[1, 1, 1, 1, 0, -1, -1, -1, -1]$ | 0 |

The policy changes during iterations $0, 1, 2, 3$. At iteration 4, the improved policy is identical to the policy that was evaluated, so the algorithm has converged.

Number of policy-changing iterations = 4

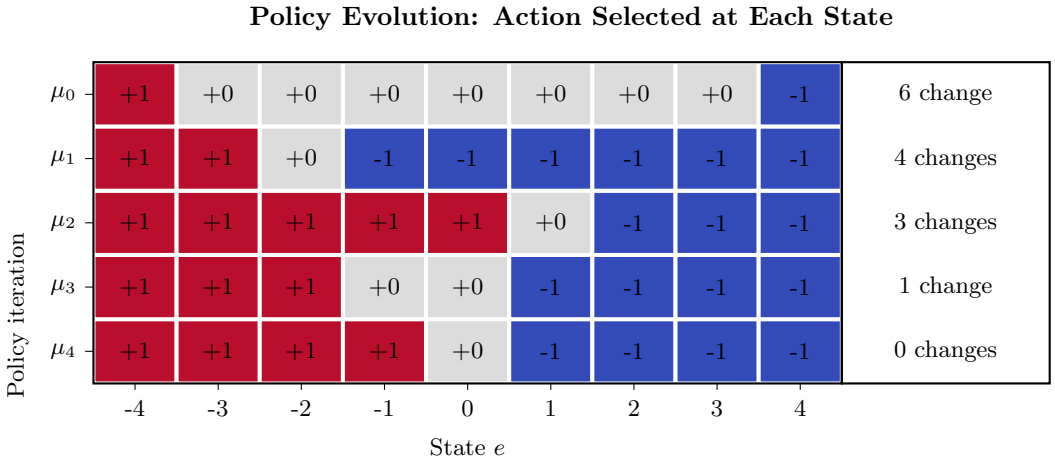



Figure 2: Policy Evolution Overall Diagram

# Problem 2 (MC/TD approximate PI)

## Part-1: Monte Carlo (MC) vs. Temporal Difference (TD(0))

We are going to evaluate MC and TD(0) using one random seed, **In this case the seed is 2026**(which is kind of interesting because most of the random seeds were giving better results for TD(0), whereas this one provides better results for MC compared to exact).

The initial policy is

$$\mu_0 = [1, 0, 0, 0, 0, 0, 0, 0, -1],$$

where the states are ordered as

$$[-4, -3, -2, -1, 0, 1, 2, 3, 4].$$

We will evaluate:

- 100 trajectories per evaluation.
- 60 transitions per trajectory.
- 6,000 transitions in total for TD(0).
- A uniformly sampled starting state from the nine-state set for each trajectory.

The same policy and transition model for both methods.

**The reference: exact $J_{\mu_0}$:**

Before comparing MC and TD, we need a reference value to measure their accuracy and this is why wee need to evaluate the Exact function.

The exact policy-evaluation equation is

$$(I - \gamma P_{\mu_0}) J_{\mu_0} = g_{\mu_0}.$$

Here:

- $P_{\mu_0}$ is the transition matrix under the initial policy.
- $g_{\mu_0}$ is the immediate cost vector.
- $\gamma = 0.95$ is the discount factor.
- $J_{\mu_0}$ contains the exact expected discounted cost for each state under the fixed policy.

And we find the values for exact policy evaluation are,

Table 4: Horizon Table of States and Exact Values

| **State** ($e$) | -4 | -3 | -2 | -1 | 0 | 1 | 2 | 3 | 4 |
|---|---|---|---|---|---|---|---|---|---|
| **Exact** $J_{\mu_0}(e)$ | 118.009897 | 110.809897 | 96.271283 | 94.279689 | 106.647348 | 129.479963 | 156.297845 | 178.622726 | 185.822726 |

The purpose of this is to see how closely MC and TD can estimate these values from sampled experience.

**Monte Carlo(MC) evaluation — learning from complete trajectories:**

Given that,

$$\widehat{J}_\theta(e) = \phi(e)^\top \theta, \qquad \phi(e) = \begin{bmatrix} 1 \\ z \\ z^2 \\ z^3 \end{bmatrix}, \quad z = \frac{e}{4}.$$

There are nine states, but we need to represent their values using just four parameters and four coefficients:

$$\theta = [\theta_0, \theta_1, \theta_2, \theta_3]^\top.$$

The estimated value at that state is a weighted combination of those four numbers. The parameters $\theta$ determine the weights.

**Algorithm 2:** How MC works

**for** *each of the* 100 *trajectories* **do**
- Start at a randomly selected state;
- Follow the fixed policy $\mu_0$ for 60 transitions;
- Record the costs along the trajectory;
- Calculate the discounted return from its starting state;
- Use that return as a training target for the starting state's features;

The return is

$$G_0 = \sum_{k=0}^{59} \gamma^k g(e_k, \mu_0(e_k)).$$

MC then fits the four parameters using ordinary least squares:

$$\theta_{\mathrm{MC}} = \arg\min_{\theta} \sum_{j=1}^{100} \left( \phi(e_0^{(j)})^\top \theta - G_j \right)^2.$$

Because trajectories are only 60 steps long and the task has no terminal state, these are finite-horizon sampled returns—not exact infinite-horizon returns. This is one reason the MC estimate may differ from $J_{\mu_0}$.

**Temporal Difference evaluation — learning after every transition:**
TD(0) uses the same value-function representation and the same sampled trajectories, but it updates the parameters after each transition.

Given that,

$$\delta_t = g(e, u) + \gamma \phi(e')^\top \theta - \phi(e)^\top \theta$$

There are three main parts:

- $g(e, u)$: the cost just observed.
- $\gamma \phi(e')^\top \theta$: the discounted estimate of the next state's future cost.
- $\phi(e)^\top \theta$: the current estimate for the state we just visited.

The TD error measures the difference between the updated one-step target and the current estimate.

If the new target is higher than the current estimate, the TD error is positive. If it is lower, the error is negative.

The parameter update is

$$\theta \leftarrow \theta + \alpha_t \delta_t \phi(e),$$

Where,

$$\alpha_t = \frac{0.1}{(1 + t/10000)^{0.6}}.$$

The learning rate $\alpha_t$ gradually decreases as more transitions are processed.
For TD(0), considered:

- Initialize $\theta = 0$.
- Process trajectories chronologically.
- Update $\theta$ after each of the 6,000 transitions.
- Bootstrap at the last step of each 60-step segment

MC waits for each trajectory's complete sampled return to fit its curve. TD learns incrementally from one-step targets that include the current estimate of the next state's value.

**Results: MC, TD(0), and exact values** The experiment was run using seed 2026, 100 trajectories of 60 transitions each, and the same sampled trajectories for both approximate methods.

```
symum@DESKTOP-BU0KT3Q UCRT64 /e/KAIST/Learning for Control Mobility/hw_02-Learning_for_Control/Problem_2 (main)
$ python MC_TD_one_seed.py
Random seed: 2026
Exact J_mu0: [118.009897 110.809897  96.271283  94.279689 106.647348 129.479963
 156.297845 178.622726 185.822726]
MC estimate: [126.01822  104.381763  95.798384  97.794978 107.898441 123.63567
 142.53356  162.119007 179.918906]
TD(0) estimate: [130.130503 108.370353 102.626687 109.000953 123.594598 142.509069
 161.845815 177.706281 186.191917]
MC RMSE: 8.501238
TD(0) RMSE: 9.992721
```

Figure 3: Single Seed Output

*Click for Code*

Table 5: Numerical Results: Exact vs. MC and TD(0) Estimates

| State ($e$) | Exact ($J_{\mu_0}$) | MC Estimate | TD(0) Estimate |
|---|---|---|---|
| $-4$ | 118.0099 | 126.0182 | 130.1305 |
| $-3$ | 110.8099 | 104.3817 | 108.3703 |
| $-2$ | 96.2713 | 95.7983 | 102.6266 |
| $-1$ | 94.2797 | 97.7949 | 109.0009 |
| 0 | 106.6473 | 107.8984 | 123.5945 |
| 1 | 129.4800 | 123.6356 | 142.5090 |
| 2 | 156.2978 | 142.5335 | 161.8458 |
| 3 | 178.6227 | 162.1190 | 177.7062 |
| 4 | 185.8227 | 179.9189 | 186.1919 |

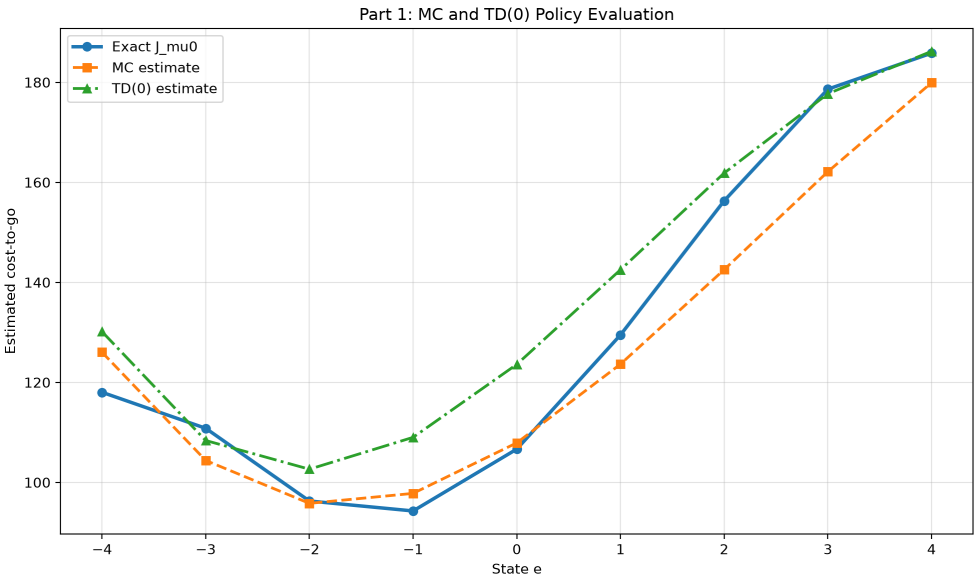



Figure 4: Caption

From the figure[3] & results MC RMSE is less than the TD(0) RMSE also shows that both methods

capture the broad trend of the value function, but their estimates differ from the exact values at some states.

## Part-2: Approximate Policy Iteration using MC and TD(0) for 5 Trials

The purpose of this part is to test whether approximate policy iteration can find an optimal policy when policy evaluation uses sampled experience rather than exact value calculations.

Each method—MC and TD(0)—must run five trials, using the same five random seeds. Every trial starts from the same initial policy $\mu_0$ and produces five policies:

$$\mu_0,\ \mu_1,\ \mu_2,\ \mu_3,\ \mu_4.$$

There are four policy improvements: $\mu_0 \to \mu_1$, $\mu_1 \to \mu_2$, $\mu_2 \to \mu_3$, and $\mu_3 \to \mu_4$.

At each evaluation, generated 100 fresh trajectories of 60 transitions each, except that the initial evaluation for seed 42 is reused from the Part 1 experiment.

Given that, the policy gap as

$$d(\mu) = \max_{e \in X} \left[ J_\mu(e) - J^*(e) \right].$$

Let's break it down:

- $J_\mu(e)$: exact expected discounted cost of following policy $\mu$ from state $e$.
- $J^*(e)$: optimal expected discounted cost from state $e$.
- $J_\mu(e) - J^*(e)$: the extra cost incurred by using policy $\mu$ instead of the optimal policy at that state.

Here, max value defines, a policy may perform well on average but still have a large error at one state. The maximum highlights that worst case.

**Output Results**

```
--- MC SUMMARY ---
Mean gaps: [152.176832  60.8       18.71014    2.32       0.      ]
Sample SD: [ 0.        0.       11.001847  5.187678  0.      ]
Final mean gap: 0.0
Final policies matching mu*: 5 / 5

--- TD SUMMARY ---
Mean gaps: [152.176832  60.8       0.         0.         0.      ]
Sample SD: [0. 0. 0. 0. 0.]
Final mean gap: 0.0
Final policies matching mu*: 5 / 5

========== EXACT PI REFERENCE ==========
mu_0 = [1, 0, 0, 0, 0, 0, 0, 0, -1], d(mu_0) = 152.176832
mu_1 = [1, 1, 0, -1, -1, -1, -1, -1, -1], d(mu_1) = 60.800000
mu_2 = [1, 1, 1, 1, 1, 0, -1, -1, -1], d(mu_2) = 26.800000
mu_3 = [1, 1, 1, 0, 0, -1, -1, -1, -1], d(mu_3) = 3.007907
mu_4 = [1, 1, 1, 1, 0, -1, -1, -1, -1], d(mu_4) = 0.000000
Optimal policy: [1, 1, 1, 1, 0, -1, -1, -1, -1]
```

Figure 5: Implementation Output

*Click for Code*

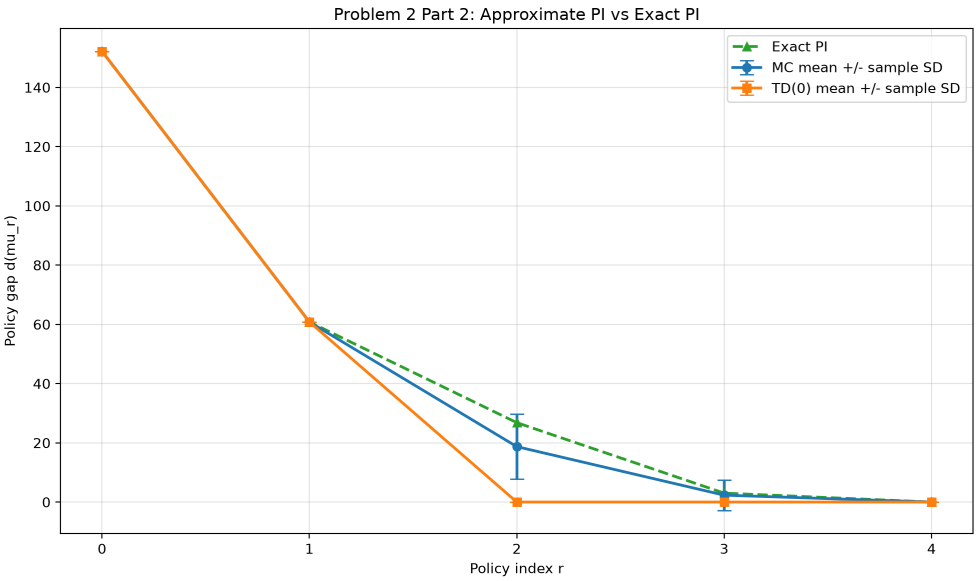



Figure 6: Policy Gap

Mean $\pm$ sample SD of $d(\mu_r)$ over 5 seeds. All methods coincide at $r = 0, 1$. TD(0) produced the same policy sequence in every trial (SD = 0). MC's lower error bar at $r = 3$ extends below zero because of one outlier trial (gap 11.6, other four 0); actual gaps are non-negative.

Comparison of suboptimality gaps $d(\mu_r)$ over policy iterations ($r$) for MC and TD methods across 5 random seeds is given below.

Table 6: Comparison of suboptimality

| | **Monte Carlo (MC)** | | **Temporal Difference (TD)** | |
|---|---|---|---|---|
| **Iteration ($r$)** | **Mean $d(\mu_r)$** | **SD** | **Mean $d(\mu_r)$** | **SD** |
| 0 | 152.18 | 0.00 | 152.18 | 0.00 |
| 1 | 60.80 | 0.00 | 60.80 | 0.00 |
| 2 | 18.71 | 11.01 | 0.00 | 0.00 |
| 3 | 2.32 | 5.19 | 0.00 | 0.00 |
| 4 | 0.00 | 0.00 | 0.00 | 0.00 |

TD(0) produced the same policy sequence in all five trials (SD = 0); its value estimates differ across seeds but not enough to change any greedy action.

Running the code to will obtain the precise means and sample standard deviations for the five trials.

Table 7: Final Optimization Results

| **Metric** | **MC** | **TD(0)** |
|---|---|---|
| Final mean policy gap | 0.0 | 0.0 |
| Final policies matching $\mu^*$ | 5/5 | 5/5 |

The key observation is that TD(0) reaches the optimal policy earlier in these trials, while MC approaches it over successive improvements. Both methods reach the optimal policy by $\mu_4$ in all five trials.

## Part-3: Explanation on the work

**(i) why more 60-step trajectories do not remove MC truncation bias, and why TD still bootstraps at the last step;**

If the trajectory is stopped after $T = 60$ transitions, the return is

$$G_t^{(60)} = \sum_{k=0}^{59} \gamma^k g_{t+k}.$$

This includes only the costs observed within the 60-step trajectory.

But the true cost-to-go also includes the remaining future costs:

$$J_\mu(e_t) = \mathbb{E}_\mu \left[ \sum_{k=0}^{59} \gamma^k g_{t+k} + \gamma^{60} J_\mu(e_{t+60}) \mid e_t \right].$$

The second term,

$$\gamma^{60} J_\mu(e_{t+60}),$$

represents the discounted expected cost after the sampled trajectory ends.

The MC return omits this term.

Therefore, even if we collect many trajectories, the returns still stop at 60 steps. Monte Carlo (MC) estimates the cost-to-go using the discounted sum of observed costs. When each trajectory is truncated after 60 transitions, the return includes only the costs observed within that horizon and omits the discounted future cost beyond the cutoff. Consequently, increasing the number of trajectories reduces **sampling error** but **does not eliminate the systematic bias caused by the fixed truncation horizon**. In this problem, the costs are nonnegative, so the expected omitted tail is nonnegative.The factor $\gamma^{60} = 0.95^{60}$ is approximately 0.0461. This discounts the tail substantially, but it does not make the tail exactly zero.

For a transition from $e_t$ to $e_{t+1}$, the TD(0) target is

$$g_t + \gamma \widehat{J}_\theta(e_{t+1}).$$

The corresponding TD error is

$$\delta_t = g_t + \gamma \widehat{J}_\theta(e_{t+1}) - \widehat{J}_\theta(e_t).$$

The estimate is updated using this error.

At the 60th transition, the target remains

$$g_{59} + \gamma \widehat{J}_\theta(e_{60}).$$

Here, step 60 is the end of the sampled trajectory, not a terminal state of the Markov DP. The process can continue from $e_{60}$, and its future costs still matter. The estimated value $\widehat{J}_\theta(e_{60})$ acts as a prediction of those future costs. TD(0), in contrast, bootstraps by using the immediate cost plus the estimated cost-to-go of the next state. Its target at the last sampled transition remains $(g_t + \gamma \widehat{J}_\theta(e_{t+1}))$, because the 60-step cutoff is an artificial sampling boundary rather than a terminal state of the MDP. The bootstrap term approximates future costs, although its accuracy depends on the current value estimate.

**(ii) whether a cost-to-go estimate with nonzero RMSE can yield an optimal greedy policy;**

We know, the cost-to-go approximation is

$$\widehat{J}_\theta(e) = \phi(e)^\top \theta,$$

where,

$$\phi(e) = \begin{bmatrix} 1 & z & z^2 & z^3 \end{bmatrix}^\top, \qquad z = \frac{e}{4}.$$

The root mean squared error (RMSE) compares the approximate cost-to-go with the exact cost-to-go:

$$\text{RMSE} = \sqrt{\frac{1}{9} \sum_{e \in X} \left( \widehat{J}_\theta(e) - J_\mu(e) \right)^2}.$$

So, as per the equation, an RMSE of zero means the estimates exactly match the reference costs at all nine states. A nonzero RMSE means at least one estimated value differs from its reference value.
So, we can say, however, RMSE measures value-estimation accuracy, not whether the greedy action choices are correct.
Now, the greedy policy chooses the action with the smallest estimated one-step cost:

$$\widehat{\mu}(e) = \arg\min_{u \in U(e)} \left[ g(e,u) + \gamma \sum_{w} P(w) \widehat{J}_{\theta}(e+u+w) \right].$$

Here, the algorithm does not need every estimated state value to be exact. It needs the action with the smallest estimated cost to be the same action that minimizes the true cost.

Also, estimation error may change the action ranking and leads to a suboptimal decision. Therefore:

- Large value errors do not necessarily imply a bad policy.
- Small value errors do not guarantee a good policy.
- What matters for greedy improvement is whether the estimated action costs preserve the correct action ordering at every state.

Therefore, exact state-value estimates are not necessary.

**(iii) for the policy-evaluation and policy-improvement stages, which use samples and which require transition probabilities. Does this small example demonstrate a computational advantage over exact PI?**

In the approximate policy iteration implementation, policy evaluation uses sampled trajectories: MC fits the cost-to-go approximation from discounted returns, while TD(0) updates it using immediate costs and estimated next-state values. The comparison between Exact Policy & Approximate PI is given below,

Table 8: Comparison between Exact Policy Iteration and Approximate Policy Iteration

| **Stage** | **Exact Policy Iteration** | **Approximate PI** |
|---|---|---|
| **Policy evaluation** | Uses transition probabilities and solves Bellman equations | Uses MC or TD(0) samples |
| **Policy improvement** | Uses transition probabilities and exact cost-to-go values | Uses transition probabilities and estimated cost-to-go values |
| **Cost-to-go accuracy** | Exact, up to numerical precision | Approximate |
| **Main computational work** | Repeated exact policy evaluations and action comparisons | Sampling, parameter updates, and action comparisons |

However, there is not any established convincing evidence, that Approximate PI is computationally cheaper than exact policy iteration.

There may have several reasons:

- The state space is very small. There are only nine states and at most three actions per state, so exact policy evaluation and improvement are inexpensive.
- Exact PI has little computational overhead in the problem. Its Bellman equations form a $9 \times 9$ linear system, which is easy to solve.
- Approximate PI also performs substantial work. The implementation uses 100 trajectories of 60 transitions per evaluation, with multiple evaluations over four improvement iterations and five trials.

- Policy improvement still uses the transition probabilities. Therefore, the experiment does not demonstrate a fully model-free approach.

So we can say, a $9 \times 9$ solve is far cheaper and noise-free compared with 6,000 transitions per evaluation. Thus, Approximate methods become more attractive in large or continuous state spaces, where constructing and solving the full Bellman system may be infeasible.
However, for more practical demonstration, a computational advantage would need to be demonstrated through measurements of runtime, sample requirements, or scalability etc. on substantially larger problems.

## References


[1] K. Choi, *Learning-Based Control for Mobility Systems: Approximate Dynamic Programming, Reinforcement Learning, and Online Lookahead*, Chapter 4, Part IV: "Approximation in Value Space,", Working draft 0.11.0-draft, Korea Advanced Institute of Science and Technology (KAIST), 08 Oct. 2026. Available: https://kyunghwan-choi.github.io/learning-based-control-mobility/learning-based-control-mobility-ch04-v0.11.0-draft.pdf.

[2] OpenAI, *ChatGPT* (GPT-5.6 Luna, default) [Large language model], 2026. Available: https://chatgpt.com.

[3] Google, *Gemini* (default model, via Google AI) [Large language model], 2026. Available: https://gemini.google.com.

[4] Anthropic, *Claude* (Claude Opus 5.5) [Large language model], 2026. Available: https://claude.ai.


## Statement of AI Tool Usage

| Tool | Provider | Purpose in this report |
|---|---|---|
| ChatGPT (GPT-5.6 Luna, default) [2] | OpenAI | Explanation of the problem statement and conceptual understanding of dynamic programming, value iteration, policy iteration, Monte Carlo (MC), TD(0), approximate policy iteration, policy gaps, and RMSE. Exploration with interpreting results and validating conceptual explanations particularly for Problem 2. |
| Gemini (default, Google AI) [3] | Google | Report writing, LaTeX formatting and document arrangement. |
| Claude (Opus 5.5) [4] | Anthropic | Python debugging and code review for identifying deviations from the specified TD(0), a multi-seed implementation of the MC/TD evaluation script, diagnostic analysis of seed-to-seed behavior. However, All outputs were run, checked, and edited by the author. |

Table 9: AI tools used in the preparation of this report.

*All AI-generated content was reviewed, and all numerical results were reproduced by running the code locally. The author takes full responsibility for the content of this report.*
*Simply open the code using VS code and click the run button; follow the Folder ordering sequence/output snapshots attached to the PDF for precise file location and all the related images and files are available at https://github.com/SymumAust/hw_02-Learning_for_Control.*